MEDICAL ERP APPLICATION ON EKS

Application
1. B2B medical ERP system
2. client: MedPharma

Challenges
1. Inventory ---> Excel
2. data spread
3. Reporting ---> single point to get result

Solution
SAS application
1. ERP solution - medical
2. centralize data manage

Technical stack
1. Frontend - React
2. Backend - Springboot, java
3. Database - MongoDB Atlas

Application Architecture
1. Frontend (s3)
2. Backend (EKS) - 3 microservices
3. Database (atlas MongoDB) -3 databases

Infrastructure
1. Frontend Flow
   mayurcbz.space --------> route53 -----> CloudFront --------> frontend(s3) ----> Application Dashboard

2. Backend Flow
   api.mayurcbz.space ----> route53 ----> LB ----> ingress ----> EKS ------> Atlas MongoDB

Team Size
1. 2 FD (frontend developer)
2. 2 BD (backend developer)
3. 1 T (Tester)
4. 1 DE (DevOps Engineer)
5. 1 PM (project manager)

Branching Strategy
1. Feature
2. Dev
3. Test
4. Uat
5. Prod
6. Hotfix

Application Link https://github.com/shubhamkalsait/edublitz-b2b-medical-erp.git

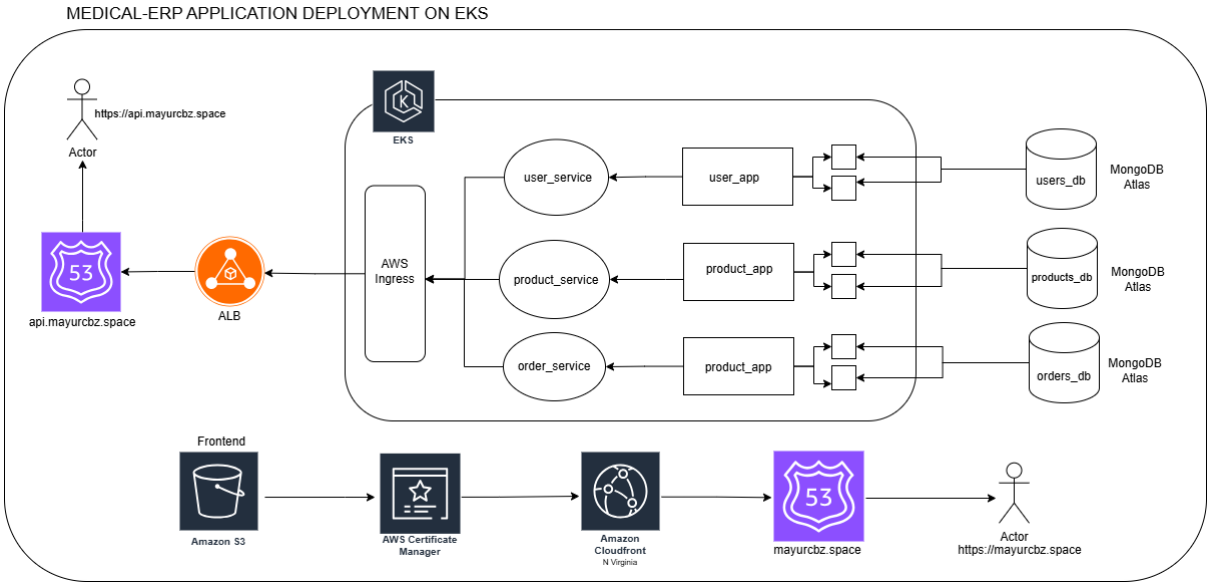
---

Prerequisite
1. database ---------> atlas account, databases
2. aws services -----> EKS, LB, Route53, S3, CloudFront, ACM, IAM

3. Domain Name ------> mayurcbz.space (Hostinger)
4. Command centre --> docker, kubectl, awscli, nodejs, npm, git

---

INFRASTRUCTURE DIAGRAM



PRACTICAL:

CONFIGURE DATABASE

1. create a cluster (edublitz)
    1. click on the cluster (edublitz)
    2. click on browse collections
    3. create 3 databases
        1. products_db
        2. orders_db
        3. users_db
    4. we want the endpoint (credentials)
        1. click on the cluster name (three dot)
        2. click on connect via -> Drivers -> copy the endpoint
    5. we want also username & password
        1. click on left panel (on MongoDB logo)
        2. click on database & Network Access
        3. click on Add new database user
        4. click on password authentication
            1. username -----> cbz
            2. password -----> redhat
        5. click on built in role -> add built in role -> atlas admin -> add user

{ add username & password in the DB endpoint url }

To connect database to the EKS -> Add the IP address in MongoDB

1. click on database and network access
2. click on ip access list
3. add ip address
    1. node-1 public-address

2. node-2 public-address
   or
3. 0.0.0.0/0

---

2. Create a EKS Cluster

1. **Attach Role to instance having EKS access**
2. **Install AWS-CLI**
   1. snap install aws-cli --classic
3. **Install kubectl**
   1. snap install kubectl --classic
   2. kubectl version --client
4. **Install eksctl**
   { https://docs.aws.amazon.com/eks/latest/eksctl/installation.html }
5. **Create A Cluster**
   1. eksctl create cluster --name eks --region us-west-2 --version 1.35 -- nodegroup-name workers --node-type c7i-flex.large --nodes 2 --managed
6. **Connect To EKS Cluster**
   1. aws eks update-kubeconfig --region us-west-2 --name eks
   2. kubectl get nodes

3.Create A Route53 Domain

1. create a hosted zone
   1. name ----> mayurcbz.space
   2. type -----> public hosted zone
   3. create a hosted zone

4.Create a S3 Bucket

1. name --------> mayurcbz.space
2. Enable -------> public-access
3. ACL ----------> acl enabled
4. create a s3 bucket

---

5. On Server

1. install docker ---> (Install Docker Engine on Ubuntu | Docker Docs)
2. install kubectl --> (snap install kubectl --classic)
3. install awscli ----> (snap install aws-cli --classic)
4. install npm ------> (apt install -y nodejs npm)
5. install git ---------> (snap install git -y)

6. Clone The Application

1. git clone https://github.com/shubhamkalsait/edublitz-b2b-medical-erp.git
2. git checkout dev

7. Adding Database Endpoints to the Services

1. Inside application (edublitz-b2b-medical-erp)

For user-service

1. cd edublitz-b2b-medical-erp/user-service/src/main/resource/
2. vim application.yaml
   Change the MongoDB URI
   mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/users_db?appName=edublitz

2. Inside application (edublitz-b2b-medical-erp)
   For product-service
   1. cd edublitz-b2b-medical-erp/product-service/src/main/resource/
   2. vim application.yaml
      Change the MongoDB URI
      mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/products_db?appName=edublitz

3. Inside application (edublitz-b2b-medical-erp)
   For order-service
   3. cd edublitz-b2b-medical-erp/ order-service/src/main/resource/
   4. vim application.yaml
      Change the MongoDB URI
      mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/orders_db?appName=edublitz

8. Creating Docker Images

1. user-service
   1. cd edublitz-b2b-medical-erp/user-service
   2. cp .env.example .env
      change the mongodb_URI
   3. docker build user-service .
   4. docker images

2.product-service
   1. cd edublitz-b2b-medical-erp/product-service
   2. cp .env.example .env
      change the mongodb_URI
   3. docker build product-service .
   4. docker images

3.order-service
   1. cd edublitz-b2b-medical-erp/order-service
   2. cp .env.example .env
      change the mongodb_URI
   3. docker build order-service .
   4. docker images

4.Frontend
   1. cd edublitz-b2b-medical-erp/frontend
   2. update the .env.production
      Change the public_ip
   3. docker build -t frontend .
   4. docker images

9.Run the Containers

1. docker network create med-erp-net
2. docker network ls

3. docker run -d -p 8081:8081 --name user-container --network med-erp-net user-service:latest

4. docker run -d -p 8082:8082 --name product-container --network med-erp-net product-service:latest
5. docker run -d -p 8083:8083 --name order-container --network med-erp-net order-service:latest
6. docker run -d -p 80:80 --name frontend-container --network med-erp-net frontend:latest

10.Run Containers using Docker-Compose file

1. cd edublitz-b2b-medical-erp/docker
2. cp .env.example .env
3. Update the .env file by adding the Mongodb URI & JWT token
4. docker compose up -d
5. docker ps

9. Creating ECR Repositories

1. order-service
2. product-service
3. user-service

10. Changing Tags & Pushing Docker Image to ECR

1. order service repo
2. product service repo
3. user service repo

11. Adding Images URI to the Deployments

1. k8s - deployment - order-service -> (image - order-service repo URI)
2. k8s - deployment - product-service -> (image - product service repo URI)
3. k8s - deployment - user-service -> (image - user service repo URI)

---

12. INSTALL AWS LOAD BALANCER CONTROLLER USING HELM

1.associate OIDC provider

```
eksctl utils associate-iam-oidc-provider \
  --cluster eks \
  --region us-west-2 \
  --approve
```

2.Verify OIDC

```
  aws iam list-open-id-connect-providers
```

3.Create IAM Policy

1.download policy

curl -o iam_policy.json https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json

2.create policy

```
aws iam create-policy \
--policy-name AWSLoadBalancerControllerIAMPolicy \
--policy-document file://iam_policy.json
```

3.verify policy

```
aws iam get-policy \
--policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy
```

4.Create IAM Role + Service Account

```
eksctl create iamserviceaccount \
  --cluster eks \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --role-name AWSLoadBalancerControllerIAMRole \
  --attach-policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve \
  --region us-west-2
```

{ change cluster-name / region / account_id }

5.Verify service account

```
kubectl describe sa aws-load-balancer-controller -n kube-system
```

6.Install Helm

1. curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
2. helm version

7.Adding Helm Repo

1. helm repo add eks https://aws.github.io/eks-charts
2. helm repo update

8.Install AWS load balancer

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=eks \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller
```

9.verify controller pods

```
kubectl get pods -n kube-system
```

---

13. Applying Manifests

1. Create Namespace
    1. cd edublitz-b2b-medical-erp/k8s/
    2. kubectl apply -f namespace.yaml

14.Configure Secrets & apply these file

```
kubectl create secret generic app-secrets \
  -n med-erp \
  --from-literal=MONGODB_URI_USER="mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/users_db?appName=edublitz" \
  --from-literal=MONGODB_URI_PRODUCT="mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/products_db?appName=edublitz" \
  --from-literal=MONGODB_URI_ORDER="mongodb+srv://cbz:redhat@edublitz.js8tilw.mongodb.net/orders_db?appName=edublitz" \
  --from-literal=JWT_SECRET="your-256-bit-hex-secret" \
  --dry-run=client -o yaml | kubectl apply -f -
```

{ copy the JWT secret from user-service --> .env.example file & paste it in the jwt_secret section in secrets}

17. Applying all the manifests

1. cd edublitz-b2b-medical-erp
    1. kubectl apply -f k8s/configmaps/
    2. kubectl apply -f k8s/deployments/
    3. kubectl apply -f k8s/services/

4. kubectl apply -f k8s/hpa/

18.Check the Deployments (working or not)

1. kubectl run mypod -it --image=ubuntu --port=80 -n med-erp
    1. apt update && apt install curl -y
    2. curl <cluster_ip_order_service>:8083/api/v1/actuator/health
       {response: status up} ---> Service is properly working

19. Creating ACM certificates

1. domain ----> api.mayurcbz.space
2. create a record in route53

19.Implementing Ingress

1. cd edublitz-b2b-medical-erp/k8s/ingress
2. vim ingress.yaml
    1. change -> alb.ingress.kubernetes.io/certificate-arn -> <Your_ACM_certificate_arn>
    2. change -> host -> <api.mayurcbz.space>
    3. remove -> remove the security group or add the sg from default vpc
3. kubectl apply -f ingressclass.yaml
4. kubectl apply -f ingress.yaml
5. kubectl get ingress -n med-erp

17.ON FRONTEND

1. vim .env.production
   VITE_USER_SERVICE_URL="https://api.mayurcbz.space/api/v1"
   VITE_PRODUCT_SERVICE_URL="https://api.mayurcbz.space/api/v1"
   VITE_ORDER_SERVICE_URL="https://api.mayurcbz.space/api/v1"

2. npm install
3. npm run build
4. aws s3 sync dist/ s3://mayurcbz.space/

18.On s3-bucket

1. Enabling static website hosting
2. copy url and hit it on web
   {You get the Application Dashboard}

19.Creating certificate in North Virginia

1. domain name -----> mayurcbz.space
2. request the certificates
3. create record in route53

20. Create a CloudFront

1. distribution-name ----> mayurcbz.space
2. route53 managed domain ----> mayurcbz.space
3. origin type -----> amazon s3
4. s3 origin --------> select bucket (mayurcbz.space)
5. TLS certificate -> <automatic_attach_Created_certificate>
6. create a distribution

21. Create a record in route53

1. create a record

2. alias ------> CloudFront-distribution
3. region ---> North Virginia
4. choose --> CloudFront distribution endpoint
5. create a record

22. Create another record in route53

1. record-name ----> api
2. alias ------> application load balancer { you can directly create CNAME record as well }
3. region ----> Oregon
4. choose ---> ALB endpoint
5. create a record

23. Edit the CloudFront Distribution

1. open CloudFront
2. select the distribution
3. go to general
4. click on edit and set
5. Default root object: index.html
6. Save

Cause: *CloudFront Default Root Object (index.html) was missing.*
Fix: *Edit the CloudFront distribution and set the Default Root Object to index.html.*

23. Links

1. mayurcbz.space --------> frontend
2. api.mayurcbz.space ---> backend

If u get login error -----> An unexpected error occurred (status: 500)

1. Check the browser Network tab (F12) and verify the login API returns 500.
2. Check the user-service logs:
    1. kubectl logs -n med-erp deployment/user-service --tail=100
3. it means there are duplicate admin users in MongoDB.
4. Open MongoDB Atlas → users_db → users collection.
5. Delete the duplicate record(s) and keep only one admin@medicalerp.com user.

Error Reason: Multiple documents with the same email (admin@medicalerp.com) exist in MongoDB, so Spring Boot fails to authenticate and returns 500 Internal Server Error.

ACCESSING APPLICATION DASHBOARD

mayurcbz.space

1.login with admin user

1. kubectl logs -n med-erp deployment/user-service | grep -i admin
    1. username: admin@medicalerp.com
    2. password: Admin@123456

2.add organisation

1. add distributor
2. type ----> distributor
3. name ---> mayur-co-pharma

4. registration number ----> abcdef-1234
5. address
6. create a distributor

3.add organisation

1. add hospital
2. type ----> hospital
3. name ---> orange city hospital
4. registration number ----> abc-123
5. address
6. create a distributor

“Once both organizations are created, the hospital can purchase medicines from the distributor."

3.Add the products

1. add products
2. distributor --- mayur-co-pharma
3. stock ---------- PARA-123-AUG-26
4. product name -- PARACETAMOL
5. Manufacturer -- zipla
6. category ------ MEDICINE
7. Unit ---------- strip
8. product type -- branded
9. MRP ---------- 20
10. Wholesale --- 15
11. create

4.Add the inventory

1. add stock (batch)
2. product ---- PARA-123-AUG-26
3. location ---- Nagpur
4. batch nu --- B2026
5. MFG ------- (optional)
6. Expiry ----- 1/1/2027
7. Quantity - 100
8. reorder alert at --- 10
9. save batch

5.Open new page in incognito mode

mayurcbz.space

1. register user
2. name ------- mayur
3. email ------- mayur@123.com
4. role --------- hospital
5. org ID ------ <copy & paste the org_id of orange-city-hospital>
6. create account

6.Get the order

1. click on order
2. distributor ---- mayur-co-pharma
3. shipping address --- Nagpur
4. line items ----- PARA-123-AUG-26
5. quantity ------- 20

6. place order

7.Go to Admin Account

1. click on orders
   { you get the status pending }
2. click on these pending
3. approve order
4. mark as dispatched

8.On user

1. confirm delivery
